Zadania rekrutacyjne do PWr Racing Team

Spis treści

1 Wstęp **2**

2 Zadanie 1 **2**
2.1 Zadanie 1.1 2
2.2 Zadanie 1.2 3

3 Zadanie 2 **4**
3.1 Zadanie 2.1 4
3.1.1 Podejrzenia co do przyczyn błędów 6
3.2 Zadanie 2.2 7
3.2.1 Identyfikacja i usuwanie anomalii z s2 8
3.2.2 Korekcja błędu s1 11
3.2.3 Program w C++ 14
3.2.4 Uwaga co do rozwiązania 14

4 Źródła **14**

1 Wstęp

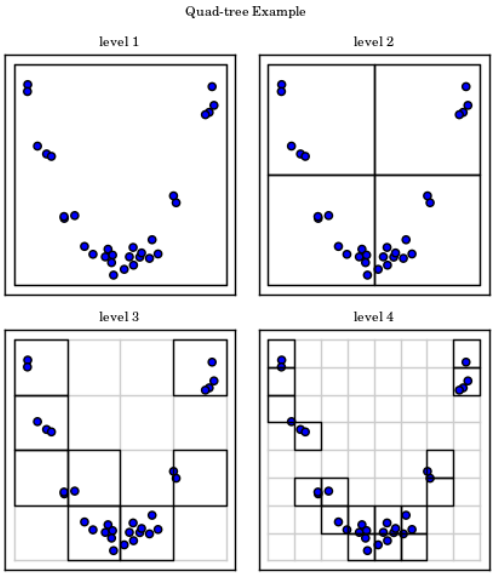
Wszystkie materiały z których korzystałem znajdują się na samym dole. W zadaniu 2.2 w kilku miejscach wspomagałem się również AI. Generowałem wykresy w Matlabie. Tam gdzie użyłem AI jest to napisane wprost.

2 Zadanie 1

W plikach źródłowych znajdują się komentarze wyjaśniające bardziej szczegółowo poszczególne fragmenty kodu

2.1 Zadanie 1.1

Pierwsze pytanie jakie sobie zadałem przy tym zadaniu to "jak zidentyfikować punkty do połączenia?". Pierwsze rozwiązanie jakie przyszło mi do głowy było najbardziej oczywiste czyli brute force - obliczanie odległości punktu od wszystkich innych punktów. Wtedy odrzuciłem to bez większego namysłu z powodu jego złożoności czasowej $O(N^2)$. Potem zacząłem myśleć nad tym aby jakoś zmapować przestrzeń 2D, podzielić ją na małe komórki (z krokiem rownym M) i sprawdzać czy w pojedynczej komórce znajduje się więcej niż jeden punkt. Poniżej wizualizacja tego co miałem na myśli.



Rysunek 1: Siatkowanie płaszczyzny 2D

Wtedy jeszcze nie zajrzalem do faktycznych danych do zadania. Kiedy jednak zobaczyłem że plik ma tylko 12 rekordów postanowiłem wybrać dużo prostsze rozwiązanie brute force ponieważ uznałem że złożoność budowania takiej siatki jest niewspółmierna do skali zadania. Na tym etapie wiedziałem jak identyfikować już punkty do scalenia. Napotkałem jednak inny problem - tworzenie zdublowanych punktów. Moje pierwsze rozwiązanie było błędne ponieważ polegało na stworzeniu wektorów które dla każdego punktu (dla każdego ID) przechowywały ID punktów które mają być z nim zmergowane. Wyglądało to tak:

ID	ID punktów do zmergowania
1	-
2	8
3	-
4	9 10
5	-
6	-
7	-
8	2
9	4 10
10	4 9
11	-
12	-

Następnie tworzyłem dodatkowe wektory które przechowywały ID punktów które mają być zmergowane ŁĄCZNIE z ID rozpatrywanego punktu. Te pomocnicze wektory wsadziłem do jeszcze jednego wektora i wyglądało to tak:

$$V = [[2, 8], [4, 9, 10], [8, 2], [9, 4, 10], [10, 4, 9]]$$

I z tego właśnie wektora tworzyłem nowe punkty więc dostawałem zdublowane wyniki (3 takie same punkty i 2 takie same punkty zamiast 2 unikalnych). Mając coś takiego postanowiłem troche obejść ten problem i posortowałem każdy z tych wektorów funkcją `std::sort` następnie posortowałem w taki sam sposób wektor tych wektorów i na końcu użyłem na nim `std::unique` aby usunąć te duplikaty. Technicznie działało ale nie podobało mi się takie rozwiązanie, więc myślałem jak pozbyć się zdublowania. W pewnym momencie przypomniałem sobie lemat o uścisku dłoni. To pociągnęło za sobą myśl o grafach. Zobaczyłem że jeśli potraktuje te punkty jako graf to wyliczone odległości każdego punktu od każdego innego punktu tworzą macierz kwadratową która w tym przypadku będzie macierzą sąsiedztwa (patrz kod źródłowy). Mając już macierz sąsiedztwa mogę użyć BFS/DFS do przeszukania grafu i zapisu grup do scalania bez ich dublowania. Moje ostateczne rozwiązanie w skrócie:

1. Oblicz długości punktu od każdego innego punktu. Wstaw 0 jeśli obliczasz długość od samego siebie, wstaw 1 jeśli punkt znajduje się wystarczająco daleko, wstaw 2 jeśli punkt znajduje sie za blisko (to tworzy macierz sąsiedztwa).

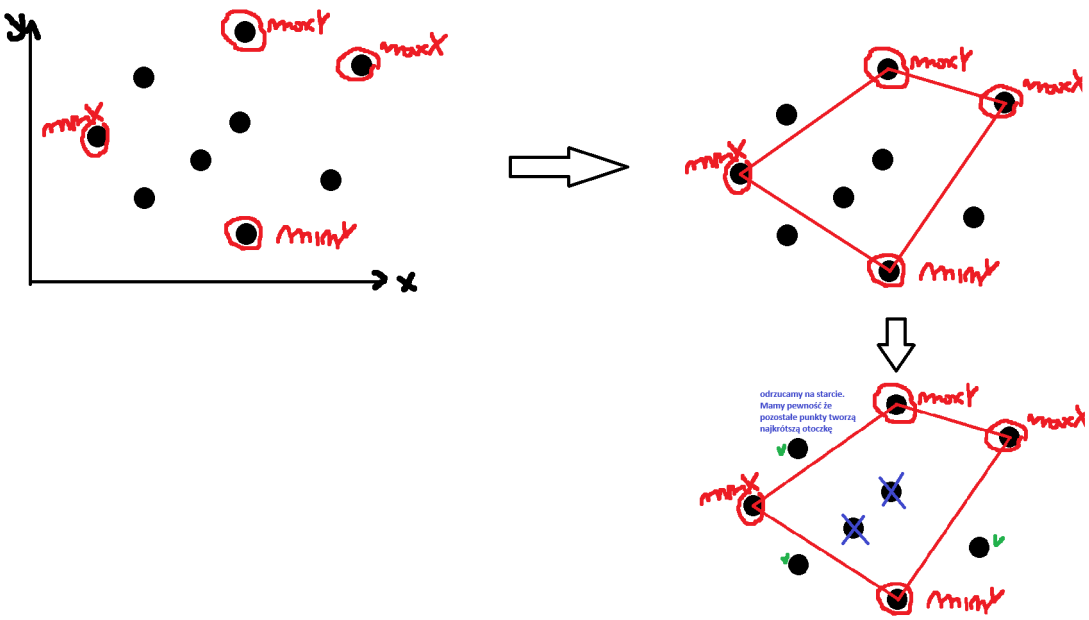
2. Dla każdego punktu w grafie wywołaj DFS aby sprawdzić czy wokół znajdują się punkty do scalenia. Jeśli tak, zapisz ID tych punktów do wektora V_1 a ten wektor zapisz do wektora V_2 (ponieważ V_1 jest tworzony dla każdej iteracji).

3. Przeiteruj po V_2 i dla każdej grupy stwórz nowe punkty

Jedyny edge case jaki przyszedł mi do głowy w tym zadaniu to sytuacja gdzie mamy takie same koordynaty punktów ale inne ID więc technicznie to różne punkty ale wtedy odległość od nich bylaby 0.0 więc i tak byłyby zmergowane. Przy obliczaniu odległości sprawdzam czy indeksy są sobie równe oraz czy odległość miedzy punktami jest mniejsza niż M. Jeśli mamy takie same punkty o różnym ID i tak byłyby zmergowane.

2.2 Zadanie 1.2

Pierwszą moją intuicją była identyfikacja punktów skrajnych w każdym kierunku (minX, maxX, minY, maxY), połączenie ich tak aby stworzyły czworokąt, a następnie odrzucenie punktów które znajdują się w środku tego czworokata. W taki sposób mamy pewność że pozostałe punkty napewno stworzą najkrótszą ścieżkę.



Rysunek 2: Wizualizacja

Problem jaki napotkałem już w kolejnym kroku planowania to chaotyczne ułożenie punktów. Nawet jeśli mam zbiór punktów które znajdą się w ścieżce, nie mogę stwierdzić w jakiej kolejności je łączyć. Próba zbudowania ścieżki stworzyła by figurę z przecieciami w srodku. Zostałem jednak przy pomyśle z odrzucaniem punktów środkowych i myślałem dalej jak wyznaczyc najkrótsza ścieżkę. Myślałem nad różnymi algorytmami najkrótszej drogi w grafie ale nie miały tutaj zastosowania. Nic nie udało mi się wymyśleć więc postanowiłem przerzucić się na inny problem - jak stwierdzić czy punkt znajduje się w środku figury 2D. Szukając w internecie natrafiłem na algorytm Jarvis march który okazał się rozwiązaniem całego zadania. Nie znałem tego algorytmu wcześniej więc musiałem się z nim zapoznać. Przy zapoznawaniu się z tym algorytmem zrozumiałem że w zasadzie mogę pozbyć się tego pierwszego kroku - wyznaczania wartości skrajnych i usuwania punktów które znajdują się w środku - ponieważ algorytm Jarvisa sam w sobie załatwia sprawę. Zdecydowałem się jednak zostawić to w kodzie ponieważ gdyby punktów było więcej i ich gęstość byłaby większa, miałoby to zauważalny wpływ na wydajność eliminując niepotrzebne porównania, a w obecnej formie nie przeszkadza. Jedyny edge jaki zauważyłem to sytuacja podczas porównywania punktów w algorytmie Jarvisa gdzie iloczyn wektorowy jest równy 0 (patrz kod źródłowy) - punkty leżą na jednej prostej. W takim wypadku biorę pod uwagę punkt dalszy bo to gwarantuje że ścieżka przejdzie równiez przez ten bliższy.

3 Zadanie 2

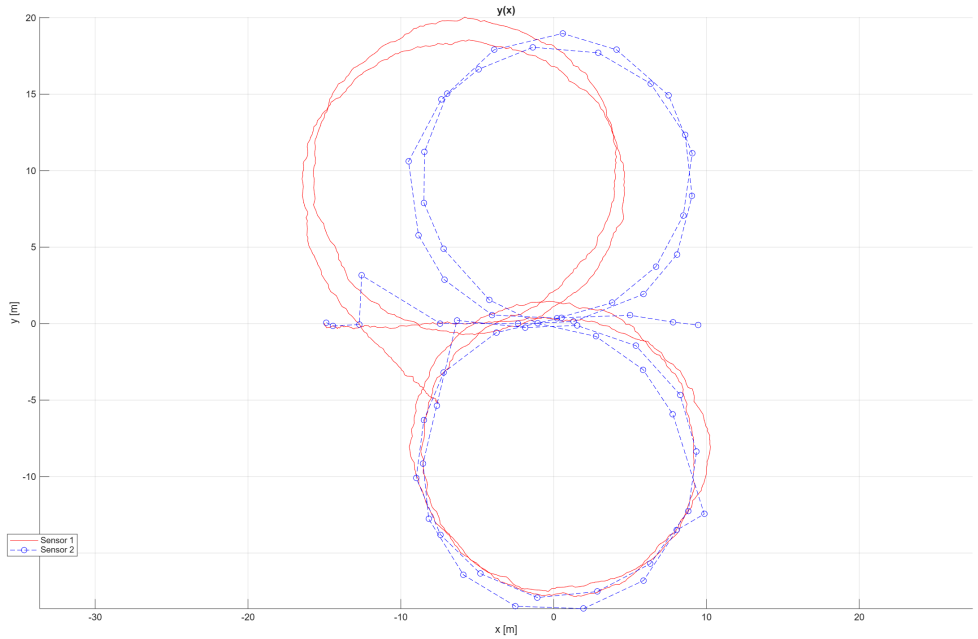
W tej części przyjmuje nazewnictwo dla uproszczenia:

- sensor1 = s1
- sensor2 = s2

3.1 Zadanie 2.1

W tym zadaniu do analizy odczytów z czujników użyłem programu Matlab. Pierwsze co zrobiłem to wykonanie 3 wykresów (dla dwóch czujników na jednej płaszczyźnie):

- $y(x)$
- $x(t)$
- $y(t)$

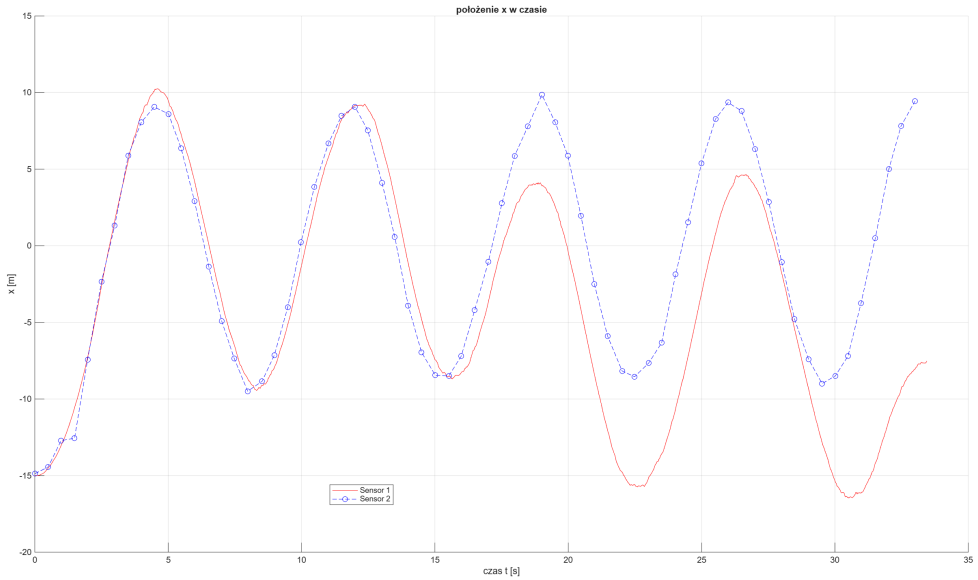


Rysunek 3: Wykresy położenia y(x) dla obu czujników

Na powyższym wykresie widać że pomiary z s2 bardziej wiernie oddają kształt toru skidpad (choć nie dość dobrze aby były użyteczne). Dodatkowo widać że s2 zarejestrował stop na współrzędnej X równej $x \approx 9.5$ podczas gdy s1 zarejestrował stop na współrzędnej X równej $x \approx -7.5$. Wiemy że bolid zatrzymał się 10 metrów za linią pomiaru czasu.

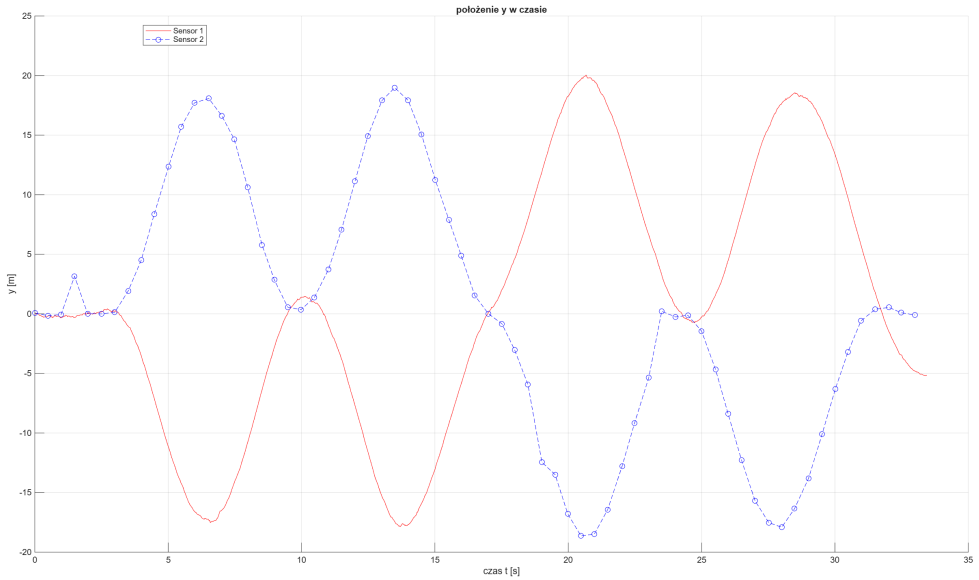
WNIOSEK 1 - s1 jest obarczony dużym błędem. Dolne okręgi s1 pokrywają się mniej więcej z odczytami z s2, jednak górne się rozjeżdżają. Zatem błąd narastał w czasie w dodatku nieliniowo - skok niedokładności jest zbyt duży. Z tego powodu dane z s1 same w sobie są zakłamane i nie nadają się do użycia.

WNIOSEK 2 - Od razu rzuca się w oczy ogromna różnica w liczbie odczytów z sensorów. s1 ma 1339 odczytów. s2 ma ich tylko 67. Dodatkowo widać anomalie, najbardziej widoczna na samym poczatku przy odczycie z s2, gwałtowny skok praktycznie pod kątem prostym. Z tych powodów nie możemy również opierać się na danych z samego s2.



Rysunek 4: Wykresy położenia x(t) dla obu czujników

Powyższy wykres dobrze ilustruje narastający w czasie błąd s1 względem s2.

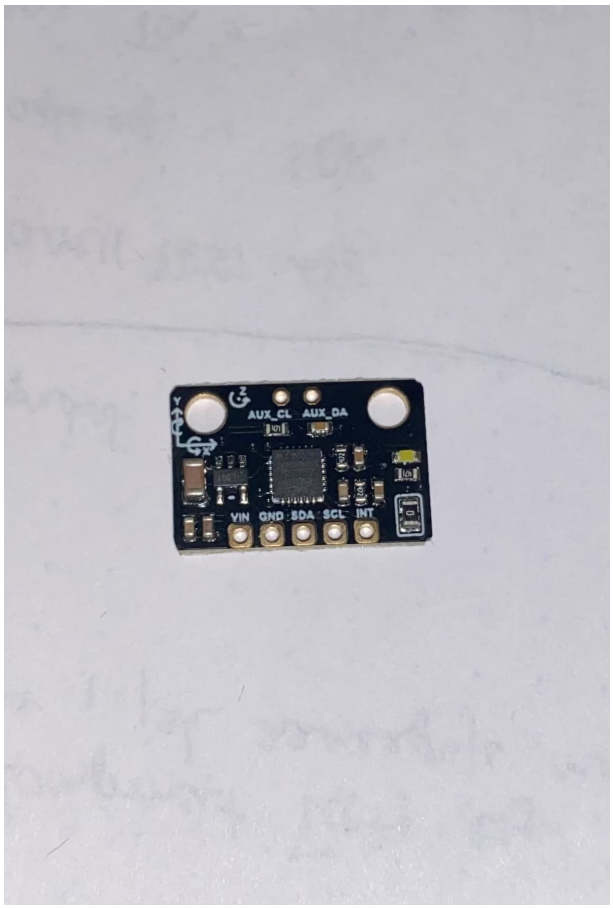


Rysunek 5: Wykresy położenia y(t) dla obu czujników

Ten wykres pokazuje kolejny błąd. Dwa czujniki były na tym samym bolidzie, podczas tego samego przejazdu, zatem powstanie takiego ułożenia się odczytów sensorów względem siebie jest niemożliwe. Odczyty osi Y jednego z czujników zostały odwrócone.

3.1.1 Podejrzenia co do przyczyn błędów

- s1 - Zważając na ilość pomiarów, narastający błąd i odwrócenie jednej z osi podejrzewam że s1 to czujnik typu IMU. Te czujniki mają ustaloną przez producenta orientację układu współrzędnych więc obrót osi o 180 stopni mógło wywołać fizyczne odwrotne zamontowanie czujnika na bolidzie. Dowodem na to niech będzie czujnik typu IMU MPU6050 który posiadam. Na poniższym zdjęciu widać jak producent definiuje układ współrzędnych dla tego czujnika. Jeśli chodzi o przyczynę narastającego błędu pomiarów, wynika to z tego że czujniki te mają akcelerometr który mierzy przyspieszenie. Aby uzykać położenie z przyspieszenia musimy całkować dwukrotnie zatem niewielki błąd pomiarowy prowadzi do szybkiej degradacji dokładności.



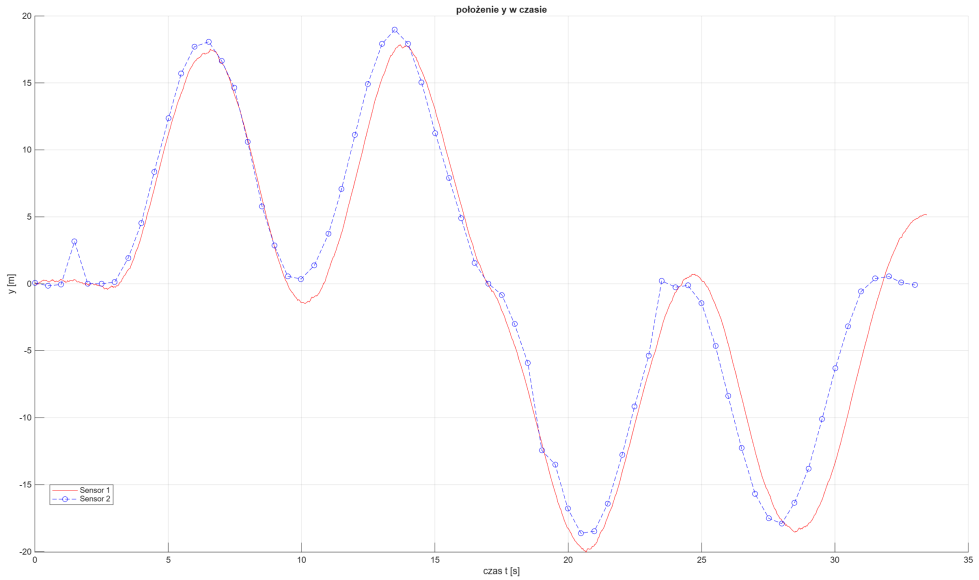
Rysunek 6: Czujnik MPU6050

- s2 - Podejrzewam że ten czujnik to czujnik GPS. Tłumaczyłoby to niską częstotliwość próbkowania oraz fakt że nie występuje tutaj błąd narastający w czasie jeśli chodzi o położenie. Nie mam pewności jednak co do tej anomalii na początku. Myślałem nad zwykłym zakłóceniem sygnału przez fizyczne przeszkody (góry, budynki, drzewa) czy może szum elektryczny (każdy czujnik będzie miał szum elektryczny, cieplny). Jednak taka drastyczna anomalia jest na samym początku. Może ma to związek z nagłym przyspieszeniem bolidu na początku?

3.2 Zadanie 2.2

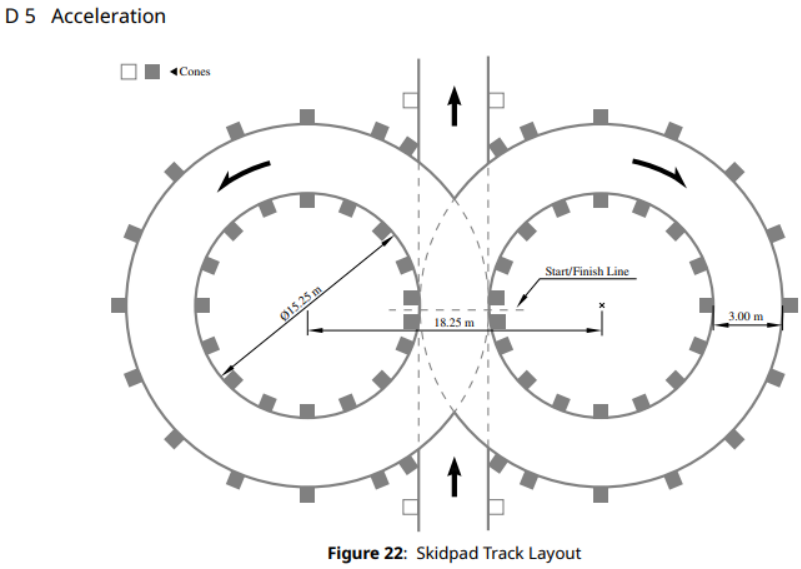
Na samym początku muszę wspomnieć że moje rozwiąznie nie spełnia jednego warunku zadania. Zadanie wymaga aby napisać program w C++ który przyjmuje pliki *sensor1.csv* oraz *sensor2.csv*. Ja jednak wykonałem procesowanie sygnałów (fuzję czujników) w Matlabie. Wymagało to zastosowania skomplikowanych funkcji interpolacji, które w Matlaba są wbudowane. Ich implementacja w C++ byłaby jednak skomplikowana. Zatem całe procesowanie sygnałów jest w Matlabie, skorygowane odczyty są zapisywane do nowego pliku csv (który znajduje się w repozytorium) i program w C++ tylko zwraca dany rekord z tego pliku. Z tego powodu umieszczam też skrypt Matlaba w repozytorium.

Pierwszym krokiem w procesowaniu sygnałów jaki zrobiłem to pomnożenie odczytów osi Y dla s1 przez -1. Wybrałem ten sensor ponieważ w oparciu o analize z zadania 2.1 zakładam że jest to czujnik IMU i prawdopodobnie był zamontowany odwrotnie. Teraz wykres $y(t)$ wygląda tak:



Rysunek 7: Wykres y(t) czujników ze skorygowanym odczytem sensora 1

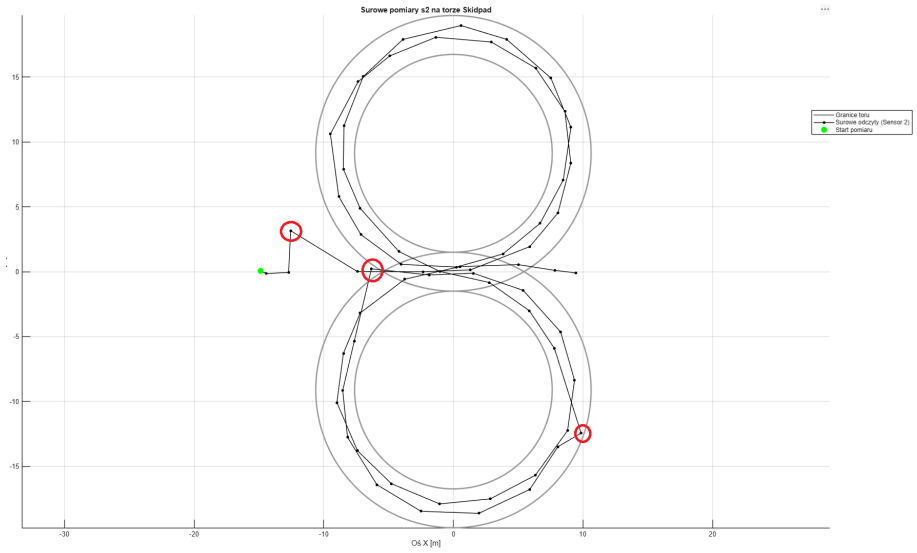
Moje rozwiązanie zakłada zastosowanie **fuzji czujników**. Zakładam że odczyty z s2 (po odszumieniu) mogą być traktowane jako "źródło prawdy" w procesie kalibracji i korygowania narastającego błędu s1. Celem będzie połączenie dokładności odszumionych pomiarów z s2 z gęstymi pomiarami s1. Przed przystąpieniem do dalszej analizy, zajrzałem do regulaminu **Formula Student Rules 2027** aby sprawdzić jakie dokładnie wymiary ma tor Skidpad. Odwzorowałem jego wymiary w Matlabie tak aby widzieć czy jakieś pomiary nie wychodzą np. poza jego granice.



Rysunek 8: Tor Skidpad

3.2.1 Identyfikacja i usuwanie anomalii z s2

Mając już rzeczywiste wymiary toru, nałożyłem na nie surowe dane z s2. Większość pomiarów wygląda sensownie pod względem fizycznym i może być bazą do korygowania s1, jednak zwróciłem uwagę na 3 punkty zaznaczone poniżej na czerwono.



Rysunek 9: Anomalie z surowych danych s2

Te punkty nie mogą być poprawnynmi pomiarami z uwagi na to jakie bolid musiałby mieć przyspieszenie w tych momentach (i gdzie musiałby się znajdować). Te punkty chcę usunąć aby nie zakłócały pomiarów. Do filtracji posłużę się sprawdzaniem przyspieszenia dośrodkowego ponieważ łatwo to skonfrontować z rzeczywistością. Jeśli gdzieś by wyszło że przyspieszenie wynosi np. 10G to wiem że należy to odrzucić. Mam dostępne położenie a chcę poznać przyspieszenie dośrodkowe. Wzór na przyspieszenie dośrodkowe:

$$a = v \cdot \omega$$

Nie mam na razie żadnego z tych składników. Wyznaczę najpierw prędkość liniową. Mogę wyliczyć różnice między położeniem $\Delta x, \Delta y$ oraz różnicę między czasem tych pomiarów Δt. Teraz z pitagorasa wyliczam długość odcinka d między dwoma pomiarami. Mając długość odcinka oraz czas mogę policzyć prędkość na tym odcinku:

$$v = \frac{d}{\Delta t}$$

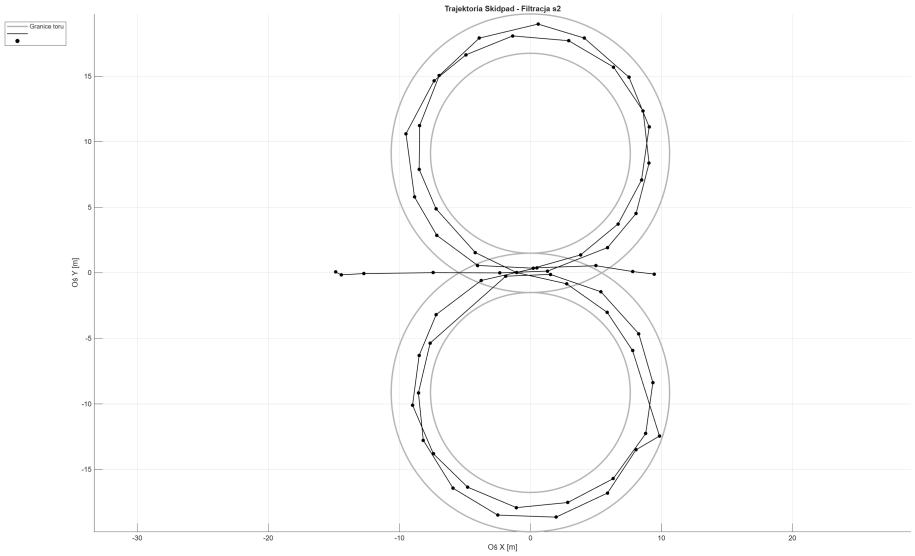
Do wyznaczeniea prędkości kątowej $\omega = \frac{\Delta\theta}{\Delta t}$ potrzebuję zmiany kąta $\Delta\theta$. Posłuzyłem się funkcją `atan2()` ponieważ znak ma tutaj znaczenie. Ze zwykłym `atan()` różnica kąta dla p1($\Delta x = 1, \Delta y = 1$) oraz p2($\Delta x = -1, \Delta y = -1$) byłaby 0 (w obu przypadkach 45 stopni). W tym zadaniu znak pokazuje kierunek jazdy więc trzeba to wziąć pod uwagę. Zatem kąt θ:

$$\theta = atan2(\Delta x, \Delta y)$$

Mam już wszystko aby móc policzyć przyspieszenie dośrodkowe:

$$a = v \cdot \frac{\Delta\theta}{\Delta t}$$

Zakładam z góry limit przyspieszenia $20 m/s^2$



Rysunek 10: Wynik filtracji

Jak widać jedna anomalia nie została usunięta. Tutaj użyłem AI do ”zdebugowania” tego problemu. Okazało się że nie wziąłem pod uwagę przyspieszenia wzdłużnego. Funkcję w Matlab która realizuje powyższe założenia (przyspieszenie wzdłużne i poprzeczne) wygenerowałem za pomocą AI

```
function [t_clean, x_clean, y_clean] = filtr_przyspieszen(t, x, y, max_a_poprzeczne, max_a_wzdluzne)
    t_clean = t;
    x_clean = x;
    y_clean = y;

    while true
        if length(t_clean) < 3
            break;
        end

        dt = diff(t_clean);
        dx = diff(x_clean);
        dy = diff(y_clean);

        % Obliczenie prędkości i kąta
        v = sqrt(dx.^2 + dy.^2) ./ dt;
        theta = atan2(dy, dx);

        % OBLICZENIA DLA PRZYSPIESZENIA POPRZECZNEGO
        dtheta = diff(theta);
        dtheta = mod(dtheta + pi, 2*pi) - pi;
        a_poprzeczne = v(2:end) .* abs(dtheta) ./ dt(2:end);

        % OBLICZENIA DLA PRZYSPIESZENIA WZDŁUŻNEGO
        dv = diff(v);
        a_wzdluzne = abs(dv) ./ dt(2:end);

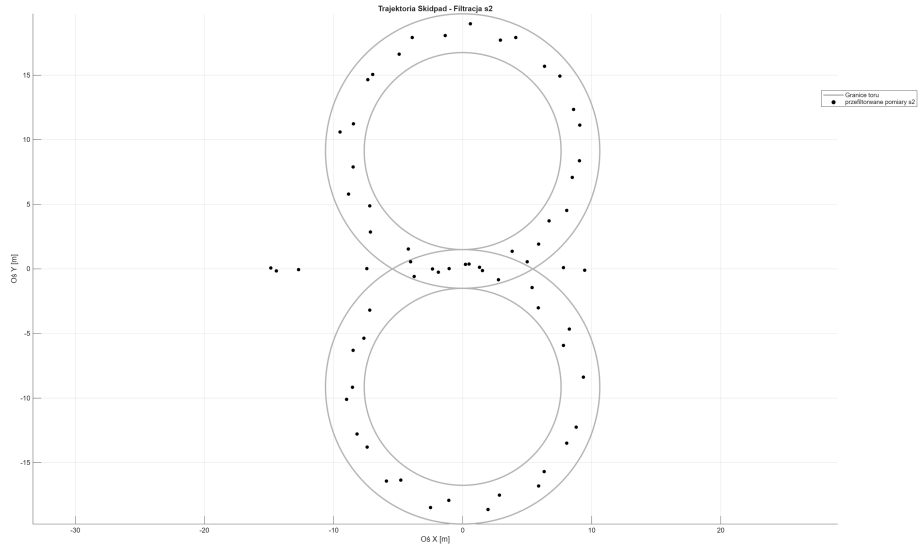
        % Szukanie największych błędów
        [max_ap_val, max_ap_idx] = max(a_poprzeczne);
        [max_aw_val, max_aw_idx] = max(a_wzdluzne);

        % Decyzja o usunięciu
        if max_ap_val > max_a_poprzeczne
            idx_do_usuniecia = max_ap_idx + 1;
            t_clean(idx_do_usuniecia) = [];
            x_clean(idx_do_usuniecia) = [];
            y_clean(idx_do_usuniecia) = [];

        elseif max_aw_val > max_a_wzdluzne
            idx_do_usuniecia = max_aw_idx + 1;
            t_clean(idx_do_usuniecia) = [];
            x_clean(idx_do_usuniecia) = [];
            y_clean(idx_do_usuniecia) = [];

        else
            break; % Koniec, brak anomalii przyspieszeniowych
        end
    end
end
```

Rysunek 11: Funkcja filtrująca pomiary s2



Rysunek 12: Wynik filtracji pomiarów s2 z uwzględnieniem przyspieszenia wzdłużnego

Na powyższym wykresie widać że filtracja odsiała pozostałą anomalię. Teraz można przystąpić do korekcji błędu s1.

3.2.2 Korekcja błędu s1

Mając odszumione pomiary s2, potraktuję je jako źródło prawdy jeśli chodzi o pozycję bolidu. Muszę zatem jakoś powiązać ze sobą pomiary s1 i s2. Mamy 2 filary na których można się oprzeć: czas, który jest absolutny oraz pomiar s2 w tym czasie (źródło prawdy). Aby skorygować błąd s1 muszę poznać uchyb względem s2. Moje rozumowanie było następujące:

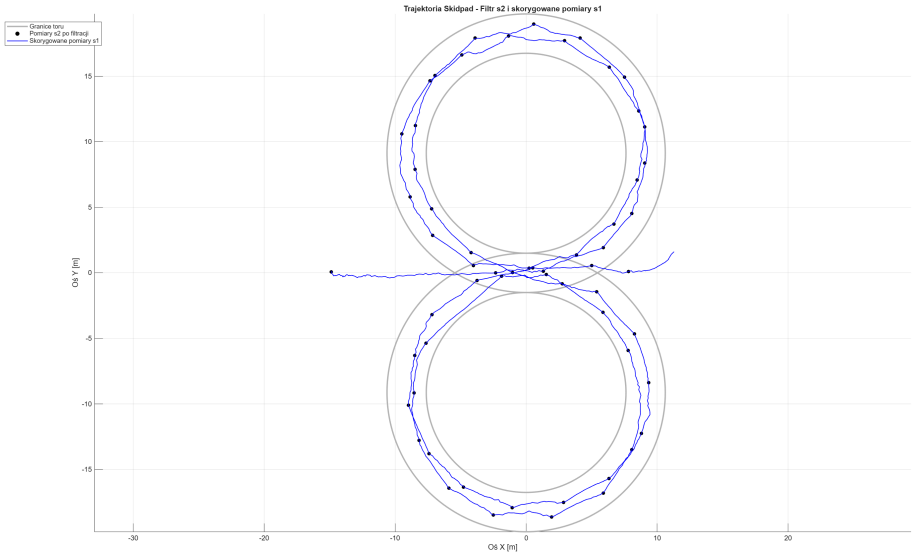
1. Biorę każdy pomiar s2 i patrzę na dwa najbliżej sąsiadujące (sąsiadujące w czasie, nie przestrzeni) pomiary s1, tuz przed pomiarem s2 i tuż po.

2. Mając te pomiary, wykonuję interpolację liniową z wagą (bo pomiar s2 nie musi trafić idealnie pomiędzy pomiary s1). W taki sposób dostaje pomiar interpolowany w czasie w którym został wykonany poprawny pomiar s2.

Tutaj pojawia się problem bo pomiarów s2 jest tylko 67. Znam już wartości uchybu s1 w tych miejscach ale pomiarów s1 jest 1339. Na początku zastosowałem interpolację liniową między tymi odczytami aby poznać wartośc uchybu dla każdego pomiaru s1. Na końcu dodałem po prostu wyliczony uchyb korygując odczyty s1. Przykład dla X ale to samo zrobiłem dla osi Y:

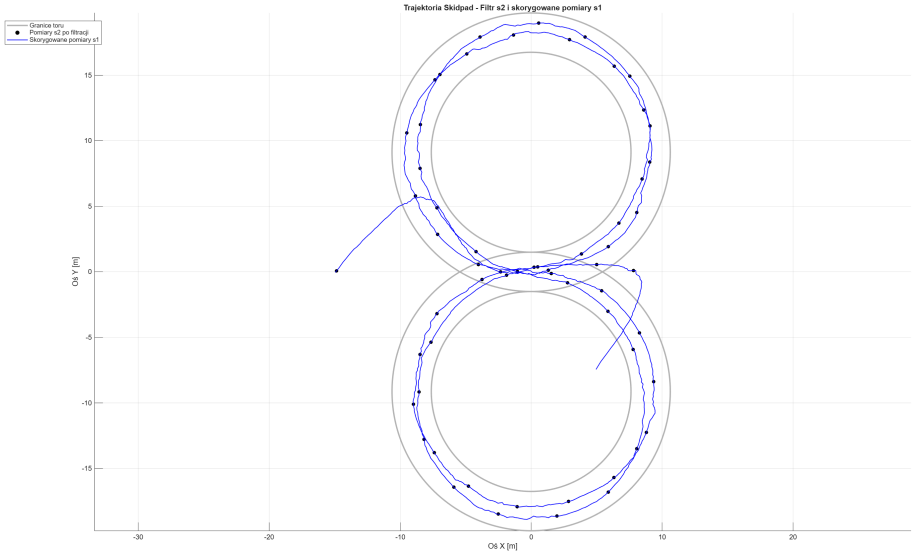
$$x_n^{skor} = x_n^{surowe} + e_n$$

Wynikiem tych operacji był poniższy wykres



Rysunek 13: Rozwiązanie z interpolacją liniową

Jak widać błąd s1 został skorygowany. Nie występuje już rozjazd. Widać jednak że to rozwiązanie nie jest idealne. Pojawiają się tu ostre krawędzie, nienaturalne zygzakowate ruchy. Po obejrzeniu kilku filmików na youtube z przejazdu toru skidpad na zawodach FS (z POV kierowcy), uznałem że taki wynik jest nieakceptowalny. Tor jest okręgiem, kierowcy trzymają kierownicę w mniej więcej stałym skręcie, zatem widoczne zygzaki nie mogą być rzeczywistym położeniem bolidu. Postanowiłem zastosować więc interpolację wielomianową za pomocą parametru `spline` zamiast `linear` w Matlabie.



Rysunek 14: Zastosowanie interpolacji spline przy korekcji s1

Widać znaczną poprawę jakości pomiarów na okręgach - teraz wygląda to rzeczywiście sensownie. Jednak pojawia się inny problem. Bolid nie jedzie tylko po okręgach. Na starcie i przy końcu jedzie w linii prostej. Zastosowanie interpolacji spline powoduje zakłamanie na tych prostych odcinkach.
Rozwiązaniem będzie w takim razie połączenie obu typów interpolacji. Fragment skryptu łączący dwa typy interpolacji wygenerowałem przy użyciu AI.

```
% Rozciągnięcie czystego błędu (Hybryda: interpolacja liniowa na końcach, makima w środku)

% Wyliczenie błędu dwiema metodami
blad_x_spline = interp1(t_s2_filtered, blad_x, t_s1, 'spline');
blad_y_spline = interp1(t_s2_filtered, blad_y, t_s1, 'spline');

blad_x_lin = interp1(t_s2_filtered, blad_x, t_s1, 'linear', 'extrap');
blad_y_lin = interp1(t_s2_filtered, blad_y, t_s1, 'linear', 'extrap');

% Ustalenie granic czasowych dla przejazdu na okregach
% Używamy drugiego i przedostatniego punktu s2, aby uniknąć anomalii brzegowych
t_start_kola = t_s2_filtered(2);
t_koniec_kola = t_s2_filtered(end-1);

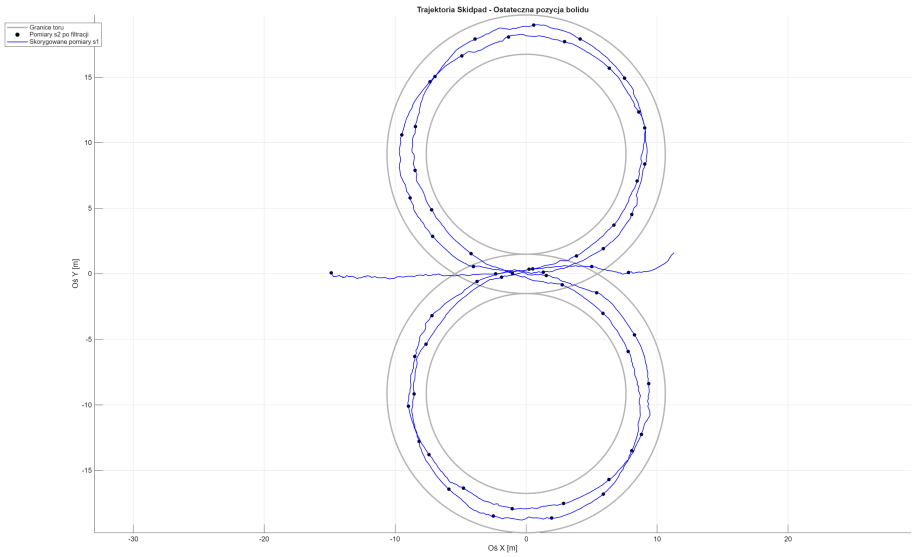
% Przygotowanie wektorów na wynik
blad_x_pelny = zeros(size(t_s1));
blad_y_pelny = zeros(size(t_s1));

% Liniowa dla prostych, Makima dla okręgu
for i = 1:length(t_s1)
    if t_s1(i) < t_start_kola || t_s1(i) > t_koniec_kola
        % Na wjeździe i zjeździe kontrujemy interpolacja błedu liniowa
        blad_x_pelny(i) = blad_x_lin(i);
        blad_y_pelny(i) = blad_y_lin(i);
    else
        % Na okręgach interpolacja błędu makima
        blad_x_pelny(i) = blad_x_spline(i);
        blad_y_pelny(i) = blad_y_spline(i);
    end
end

x_skor = x_s1 + blad_x_pelny;
y_skor = y_s1 + blad_y_pelny;
```

Rysunek 15: Fragment skryptu łączący obie interpolacje

Ostateczny wynik prezentuje się tak:



Rysunek 16: Finalny wynik fuzji czujników

Zadanie wymaga podania pozycji bolidu z dokladnością do milisekundy, zatem na ostatnim krokiem będzie jeszcze jedna interpolacja w siatce z krokiem 1ms. Takie dane będą wejściem programu w C++.

```
% Interpolacja na danych (siatka z krokiem 1ms)
t_mili = (0.000 : 0.001 : 33.000)'; % zakres podany w zadaniu
x_mili = interp1(t_s1, x_skor, t_mili, 'spline', 'extrap');
y_mili = interp1(t_s1, y_skor, t_mili, 'spline', 'extrap');
```

Rysunek 17: Interpolacja końcowa danych

3.2.3 Program w C++

Z racji na cały processing sygnałów w Matlabie, program C++ jest dość prosty. Wczytuje on plik ze skorygowaną pozycją bolidu, wrzuca go do hash mapy i czeka na input użytkownika. Wybrałem hash mapę z powodu ilości danych (33 tysiące). Hash mapa oferuje szybkie wyszukiwanie co jest jedynym celem tego programu.

3.2.4 Uwaga co do rozwiązania

Próbowałem w tym zadaniu zastosować **filtr Kalmana**. Nigdy wcześniej jednak tego nie robiłem, po prostu wiedziałem że stosuje się go w fuzji czujników. Moja implementacja jednak nie przynosiła żadnych skutków (wynikiem był poszarpany we wszystkie strony wykres) i na moment kiedy to piszę nadal nie wiem co może być przyczyną. Brakuje mi głębszej wiedzy w tym temacie i możliwe że coś przeoczyłem, dlatego zostaje przy rozwiązaniu które i tak daje sensowny wynik.

4 Źródła

- https://www.geeksforgeeks.org/dsa/convex-hull-using-jarvis-algorithm-or-wrapping/
- https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/
- https://www.formulastudent.de/fileadmin/user_upload/all/2027/rules/FS_Rules_2027_v1.0.pdf
- https://www.youtube.com/watch?v=6qV3YjFppuc&list=PLn8PRpmsu08ryYoBpEKzoMOveSTyS-h4a
- https://www.youtube.com/watch?v=0JrYTW-N2BA
- https://www.youtube.com/watch?v=pBtqaK0PzrA
- https://www.youtube.com/watch?v=s_9InuQAx-g